

Method Overloading in C# with Examples

In this article, I am going to discuss **What is Method Overloading in C#** and what is the use of method overloading in our application development with Examples. Please read our previous article before proceeding to this article where we discussed the basics of **Polymorphism in C#**. At the end of this article, you will have a very good understanding of the following pointers related to Method Overloading.

1. **What is Method Overloading in C#?**
2. **When should we Overload Methods?**
3. **What are the advantages of using Method Overloading in C#?**
4. **When is a method Considered an overloaded method?**
5. **What is Inheritance Based Method Overloading?**
6. **Real-Time Scenarios where you need to use Method Overloading?**
7. **What is Constructor Overloading in C#?**
8. **Real-Time Example Logging using Method Overloading in C#?**

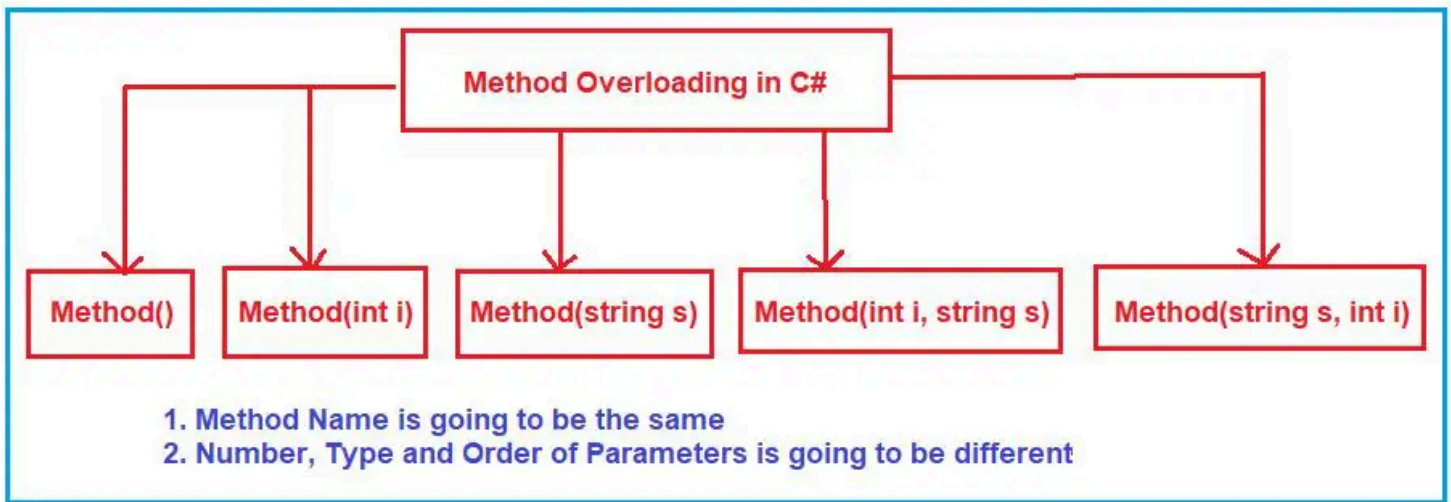
Note: The point that you need to keep in mind is function overloading and method overloading terms are interchangeably used. Method overloading is one of the common ways to implement Compile-Time Polymorphism in C#.

What is Method Overloading or Function Overloading in C#?

Method Overloading means it is an approach to defining multiple methods under the class with a single name. So, we can define more than one method with the same name in a class. But the point that you need to remember the parameters of all those methods should be different (different in terms of number, type, and order of the parameters).

So, if we are defining multiple methods with the same name but with a different signature in a class or in the Parent and Child Classes, then it is called Method Overloading in C#. That means C#.NET not only allows method overloading in the same class but also allows method overloading in Parent and Child classes. So, we can create a method in the Derived/Child class with the same name as the method name defined in the Base/Parent class in C#.

In simple words, we can say that the Method Overloading in C# allows a class to have multiple methods with the same name but with a different signature. The functions or methods can be overloaded based on the number, type (int, float, etc), order, and kind (Value, Ref or Out) of parameters. For a better understanding, please have a look at the below image. All the Methods are going to be valid and based on the method call, the compiler will automatically decide which overloaded version to be invoked.



As you can see in the above image, all the methods are having the same name i.e. Method but with different parameters. If you look at the first two methods the number of parameters is different. The first method takes zero parameters while the second method takes one parameter. Then if you compare the second method with the third method, both are taking the same number of parameters but of different types. The second method takes an integer parameter while the Third method takes a string parameter. Further, if you compare the fourth and fifth method, both are having the same number of parameter but the order of the parameters are different. The fourth method takes the first parameter as an integer and the second parameter as a string while the Fifth method takes the first parameter as a string and the second parameter as an integer. So, every method is different in terms of number, type, and order of parameters, and this is called method overloading in C#.

The signature of a method consists of the name of the method and the data type, number, order, and kind (Value, Ref or Out) of parameters. The point that you need to keep in mind is that the signature of a method does not include the return type and the params modifiers. So it is not possible to overload a method just based on the return type and params modifier. We will discuss Params modifier in our upcoming article.

Example to Understand Method Overloading in C#:

```
using System;
namespace MethodOverLoading
{
    class Program
    {
        static void Main(string[] args)
        {
            Program obj = new Program();
            obj.Method(); //Invoke the 1st Method
            obj.Method(10); //Invoke the 2nd Method
            obj.Method("Hello"); //Invoke the 3rd Method
            obj.Method(10, "Hello"); //Invoke the 4th Method
            obj.Method("Hello", 10); //Invoke the 5th Method
            Console.ReadKey();
        }
        public void Method()
        {
            Console.WriteLine("1st Method");
        }
    }
}
```

```

    public void Method(int i)
    {
        Console.WriteLine("2nd Method");
    }
    public void Method(string s)
    {
        Console.WriteLine("3rd Method");
    }
    public void Method(int i, string s)
    {
        Console.WriteLine("4th Method");
    }
    public void Method(string s, int i)
    {
        Console.WriteLine("5th Method");
    }
}

```

Output:

```

1st Method
2nd Method
3rd Method
4th Method
5th Method

```

Why Return Type is not considered as part of Method Overloading in C#?

Let us understand why return type is not considered as part of method overloading with an example. Please have a look at the following image. Here, I have written two methods with the same name but one method's return type is void, and the other method's return type is a string. See, as soon as we create the second method, the compiler itself gives the compile time error saying **Type 'Program' already defines a member called 'Method' with the same parameter types**.

```

public void Method()
{
    Console.WriteLine("1st Method");
}
public string Method()
{
    Console.Write
    return "Some

```

string Program.Method()

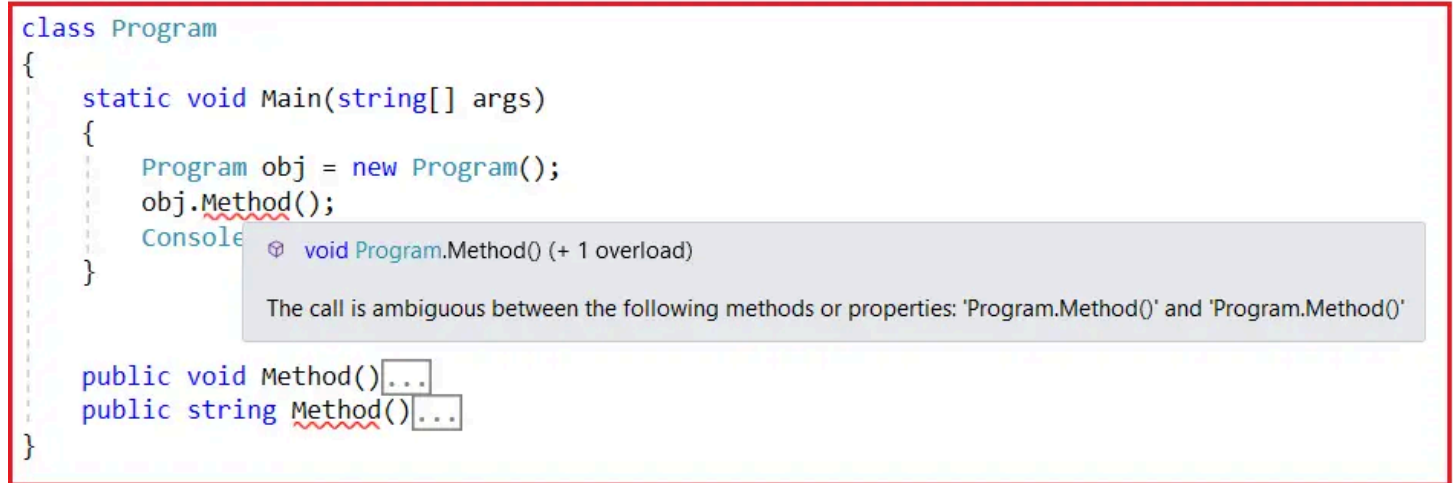
Type 'Program' already defines a member called 'Method' with the same parameter types

So, at the time of defining the method, only the compiler gave us an error. Now, still, you may have doubt, return types are different, then why it is going to be invalid. To understand, let us try to invoke the Method as shown in the below image. So, when we invoke the method, can you tell me which version of the Method is

going to be invoked? Because we have two methods that do not take any parameter. So, here we will get the ambiguity problem and see the compiler also giving the same ambiguity error **The call is ambiguous between the following methods or properties: 'Program.Method()' and 'Program.Method()'** while invoking the method.

```
class Program
{
    static void Main(string[] args)
    {
        Program obj = new Program();
        obj.Method();
        Console.WriteLine("Hello, World!");
    }
}

public void Method()
public string Method()
```



Still, you have doubts, the return types are different. See, the return types come into the picture at the end of the method execution. But here, the confusion is not at the end of the method execution, but the confusion is about where to start, and which method to invoke. So, the compiler does not have any clarity to start the method execution, and talking about the end of the method execution does not make any sense. So, this is the reason why return type is never taken into consideration while defining method overloading in C#.

When should we Overload Methods in C#?

We have understood what is Method Overloading and how to implement the Method Overloading in C#. But, the important question is when we need to implement or when we need to go for Method Overloading in C#. Let us understand this with an example.

The concept of Method Overloading falls under the Polymorphisms OOPs principle. Object Oriented Programming is based on four principles i.e. Encapsulation, Abstraction, Inheritance, and Polymorphism.

What is Polymorphism? Polymorphism is a mechanism of changing the behavior based on the inputs. That means when the input changes, automatically the output or behavior changes. The best example of polymorphism is ourselves. For example, when we hear something interesting or something which is good for us, we feel happy. And when we hear something which is not good for us, we feel sad. Suppose, you asked your father to purchase a bike, and if your father purchases a bike for you then you will feel happy. And if your father says that I am not going to purchase a bike for you, then you will feel sad. So, you are the same person, when you received something good, you feel happy and when you receive something which is not good, you feel sad. This is called polymorphism. Behaving in different ways based on the input received i.e. whenever the input changes the output automatically changes.

Here, the input changes mean don't think that the values changes. Input changes mean when we change the number, type, and order of input the values are going to be changed. Don't think that, if I pass 10, I will get a value, if I pass 20, I will get a different value. For this, if else condition is sufficient, overloading is not required. When you expect the output to be changed based on the number, type, and order of inputs, then only you need to go for Method overloading in C#.

For a better understanding, please have a look at the following example. Here, we have created three methods with the same name to perform the addition of two integers, two floats, and two strings. So, when

we give two integer numbers we will get one output and when we provide two string values, then we will get a different output, and similarly, when we give two float numbers we will get another output. That means when the input changes the output or behavior also automatically changes. This is called polymorphism in C#.

```
using System;
namespace MethodOverLoading
{
    class Program
    {
        public void Add(int a, int b)
        {
            Console.WriteLine(a + b);
        }
        public void Add(float x, float y)
        {
            Console.WriteLine(x + y);
        }
        public void Add(string s1, string s2)
        {
            Console.WriteLine(s1 + " " + s2);
        }
        static void Main(string[] args)
        {
            Program obj = new Program();
            obj.Add(10, 20);
            obj.Add(10.5f, 20.5f);
            obj.Add("Pranaya", "Rout");
            Console.ReadKey();
        }
    }
}
```

Output:

A screenshot of a terminal window showing the output of the C# program. The output consists of three lines: '30' on the first line, '31' on the second line, and 'Pranaya Rout' on the third line. The text is white on a black background.

```
30
31
Pranaya Rout
```

Suppose you are the user of the Program class and when you create the Program class instance and when you type the object name and dot operator, then you will not see three different Add methods, rather you will see only one Add method with two overloaded versions of the Add method as shown in the below image.

```

class Program
{
    public void Add(int a, int b) [...]
    public void Add(float x, float y) [...]
    public void Add(string s1, string s2) [...]
    static void Main(string[] args)
    {
        Program obj = new Program();
        obj.
    }
}

```

void Program.Add(int a, int b) (+ 2 overloads)

Add
Equals
GetHashCode
GetType
MemberwiseClone
ToString

So, the advantage of using Method overloading is that if we overload the methods, then the user of our application gets a comfortable feeling in using the method with an impression that he/she calling one method by passing different types of values. The best example for us is the system-defined **"WriteLine()"** method. It is an overloaded method, not a single method taking different types of values. If you go to the definition of the Console class, then you will see the following overloaded versions of the WriteLine method defined inside the Console class.

```

...public static void WriteLine(float value);
...public static void WriteLine(int value);
...public static void WriteLine(uint value);
...public static void WriteLine(long value);
...public static void WriteLine(ulong value);
...public static void WriteLine(object value);
...public static void WriteLine(string value);
...public static void WriteLine(string format, object arg0);
...public static void WriteLine(string format, object arg0, object arg1, object arg2);
...public static void WriteLine(string format, object arg0, object arg1, object arg2, object arg3);
...public static void WriteLine(string format, params object[] arg);
...public static void WriteLine(char[] buffer, int index, int count);
...public static void WriteLine(decimal value);
...public static void WriteLine(char[] buffer);
...public static void WriteLine(char value);
...public static void WriteLine(bool value);
...public static void WriteLine(string format, object arg0, object arg1);
...public static void WriteLine(double value);

```

When is a method considered an overloaded method in C#?

If two methods have the same method name but with different signatures, then those methods are considered overloaded methods. Then the rule we should check is both methods must have different parameter **Types/Numbers/Orders**. But there is no rule on return type, access specifier and access modifier

means overloading methods can have their own return type (void, float, int, string, etc), own access specifier (public, private, protected, etc.), and access modifier (sealed, static, virtual, etc.) because overloading methods are different methods

Can we Overload Methods in the Same Class?

Yes, it is possible. No Compile Time Error, and No Runtime Error. Methods can be overloaded in the same or in super and sub classes because overloaded methods are different methods. But we can't override a method in the same class it leads to Compile Time Error: **"Method is Already Defined"** because overriding methods are the same methods with a different implementation. In our next article, we are going to discuss [Method Overriding in C#](#) with Examples.

What is Inheritance-Based Method Overloading in C#?

A method that is defined in the parent class can also be overloaded under its child class. It is called Inheritance Based Method Overloading in C#. See the following example for a better understanding. As you can see in the below code, we have defined the Add method twice in the class Class1 and also defined the Add method in the child class Class1. Here, notice every Add method takes different types of parameters.

```
using System;
namespace MethodOverLoading
{
    class Class1
    {
        public void Add(int a, int b)
        {
            Console.WriteLine(a + b);
        }
        public void Add(float x, float y)
        {
            Console.WriteLine(x + y);
        }
    }
    class Class2 : Class1
    {
        public void Add(string s1, string s2)
        {
            Console.WriteLine(s1 + " " + s2);
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            Class2 obj = new Class2();
            obj.Add(10, 20);
            obj.Add(10.5f, 20.7f);
            obj.Add("Pranaya", "Rout");
            Console.ReadKey();
        }
    }
}
```

```
}  
}
```

Output:

```
30  
31.2  
Pranaya Rout
```

Note: To overload a parent class method under its child class the child class does not require any permission from its parent class.

Real-Time Scenario where you Can use Method Overloading in C#

Suppose you are working on a maintenance project. And you are going to work on a class where already some parameterized constructors have been defined and you need to pass some additional parameters. So what you will do, whether you will add the required parameter with one of the already defined constructors or add a new constructor as per your requirement? In such cases, you should not add the required parameter with the already defined constructor because this may disturb your other class dependency structure. So what you will do is create a new constructor with the required parameter. That new constructor that you are creating is nothing but the Constructor Overloading in C#.

Example to Understand Constructor Method Overloading in C#

Please have a look at the following example. Here, we are creating three different versions of the Constructor, and each constructor takes a different number of parameters, and this is called Constructor Overloading in C#.

```
using System;  
namespace ConstructorOverloading  
{  
    class ConstructorOverloading  
    {  
        int x, y, z;  
        public ConstructorOverloading(int x)  
        {  
            Console.WriteLine("Constructor1 Called");  
            this.x = 10;  
        }  
        public ConstructorOverloading(int x, int y)  
        {  
            Console.WriteLine("Constructor2 Called");  
            this.x = x;  
            this.y = y;  
        }  
        public ConstructorOverloading(int x, int y, int z)  
        {  
            Console.WriteLine("Constructor3 Called");  
            this.x = x;  
        }  
    }  
}
```

```

        this.y = y;
        this.z = z;
    }
    public void Display()
    {
        Console.WriteLine($"X={x}, Y={y}, Z={z}");
    }
}
class Test
{
    static void Main(string[] args)
    {
        ConstructorOverloading obj1 = new ConstructorOverloading(10);
        obj1.Display();
        ConstructorOverloading obj2 = new ConstructorOverloading(10, 20);
        obj2.Display();
        ConstructorOverloading obj3 = new ConstructorOverloading(10, 20, 30);
        obj3.Display();
        Console.ReadKey();
    }
}

```

Output:

```

Constructor1 Called
X=10, Y=0, Z=0
Constructor2 Called
X=10, Y=20, Z=0
Constructor3 Called
X=10, Y=20, Z=30

```

Method Overloading Realtime Example using C# Language:

Suppose, you are working on a real-time project. One of the mandatory things that we need to do is logging. Whenever a request comes, when a method call, we need to capture the request details, we need to log the necessary information either on a text file or in the database. Even when some exceptions occurred we also need to log the exception detail so that later we can verify the log and need to identify the reasons for the exception. In the below example, we are creating a class called Logger with many overloaded versions of the Log method. So, as per our requirement, we can call the appropriate method.

```

using System;
namespace MethodOverLoading
{
    public class Logger
    {
        public static void Log(string ClassName, string MethodName, string Message)
        {

```

```

        Console.WriteLine($"DateTime: {DateTime.Now.ToString()}, ClassName: {ClassName}, MethodName: {MethodName}, Message: {Message}");
    }
    public static void Log(string uniqueId, string ClassName, string MethodName, string Message)
    {
        Console.WriteLine($"DateTime: {DateTime.Now.ToString()}, UniqueId: {uniqueId}, ClassName: {ClassName}, MethodName: {MethodName}, Message: {Message}");
    }
    public static void Log(string Message)
    {
        Console.WriteLine($"DateTime: {DateTime.Now.ToString()}, Message: {Message}");
    }
    public static void Log(string ClassName, string MethodName, Exception ex)
    {
        Console.WriteLine($"DateTime: {DateTime.Now.ToString()}, ClassName: {ClassName}, MethodName: {MethodName}, Exception Message: {ex.Message}, \nException StackTrace: {ex.StackTrace}");
    }
    //You create many overloaded versions as per your business requirement
}

class Program
{
    static void Main(string[] args)
    {
        string ClassName = "Program";
        string MethodName = "Main";
        string UniqueId = Guid.NewGuid().ToString();
        Logger.Log(ClassName, MethodName, "Message 1");
        Logger.Log(UniqueId, ClassName, MethodName, "Message 2");
        Logger.Log("Message 3");
        try
        {
            int Num1 = 10, Num2 = 0;
            int result = Num1 / Num2;
            Logger.Log(UniqueId, ClassName, MethodName, "Message 4");
        }
        catch (Exception ex)
        {
            Logger.Log(ClassName, MethodName, ex);
        }

        Console.ReadKey();
    }
}

```

Output:

```
DateTime: 20-08-2022 10:27:03, ClassName: Program, MethodName:Main, Message:Message 1
DateTime: 20-08-2022 10:27:03, UniqueId: 6793a9ad-2f8d-41ab-bfa3-2df2e03b7ab5, ClassName: Program, MethodName:Main, Message:Message 2
DateTime: 20-08-2022 10:27:03, Message: Message 3
DateTime: 20-08-2022 10:27:03, ClassName: Program, MethodName:Main, Exception Message:Attempted to divide by zero.,
Exception StackTrace:    at MethodOverloading.Program.Main(String[] args) in C:\Users\HP\source\repos\MethodOverloading\MethodOverloading\Program.cs:line 40
```

In the next article, I am going to discuss [Operator Overloading in C#](#) with Examples. Here, in this article, I try to explain What exactly Method Overloading is in C# and when and how to use **Method Overloading in C#** with Examples. I hope you enjoy this Method Overloading in C# with Examples article.